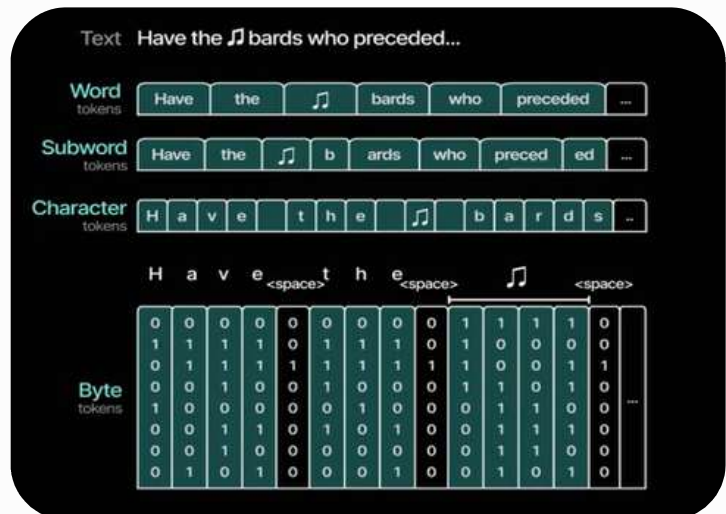




0:00

In this lecture, we'll explore

0:02

tokenization. The process of converting

0:05

text into smaller units called tokens

0:08

before sending them to a language model.

0:10

Show more ▾

The explanation in the transcript is **good for beginners**, but it simplifies a few things that are technically inaccurate. Here's a more accurate explanation of the entire process.

ChatGPT ▾ [Upgrade](#)

[Share](#) ...

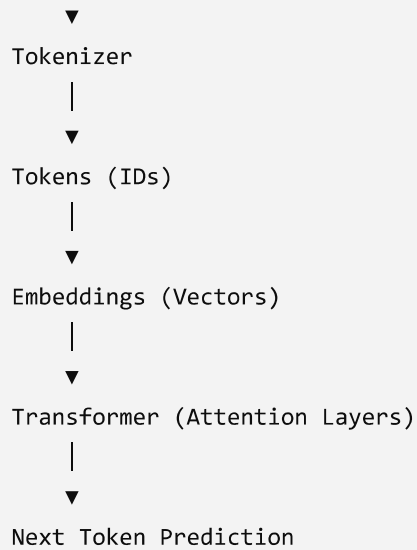
What is Tokenization?

Tokenization is the **first step** before a Large Language Model (LLM) processes your prompt.

It converts human-readable text into **tokens**, which are numerical IDs that the model understands.

Your Text





Example

Input:

Have the 🎵 bards who preceded us



Depending on the tokenizer, this can become different kinds of tokens.

1. Word Tokenization

Entire words become tokens.

Have | the | 🎵 | bards | who | preceded | us



Pros

- Simple
- Easy to understand

Cons

- Huge vocabulary
- Unknown words become difficult

Example

Vocabulary



Have → 501

the → 12

bards → 874
preceded → 4312

2. Subword Tokenization (Used by GPT, Llama, Gemini)

Instead of storing every word, the tokenizer stores frequent pieces of words.

Example

Have
the
🎵
b
ards
who
preced
ed
us



Notice

preceded



↓

preced + ed

or

pre
ced
ed



depending on the tokenizer.

This is why the screenshot shows

Have
the
🎵
b
ards
who



preced
ed

Different models split differently.

Why split words?

Imagine these words

play

playing

played

player

playful



Instead of storing every word separately

Vocabulary

play
ing
ed
er
ful



Now the tokenizer can build many words from fewer pieces.

This dramatically reduces vocabulary size.

3. Character Tokenization

Every character becomes one token.

H
a
v
e

t
h



e



b

a

r

d

s

Pros

Works with every language.

Cons

Very long sequences.

Example

Hello



↓

H e l l o

5 tokens

4. Byte Tokenization

Instead of characters, text becomes bytes.

Every computer stores text as bytes.

Example

A



↓

65

↓

01000001

Emoji



becomes multiple bytes

```
11110000
10011111
10001110
10110101
```



This allows models to process

- emojis
- Chinese
- Arabic
- Hindi
- Symbols
- Any Unicode text

without needing a separate vocabulary.

GPT-2 introduced byte-level BPE for this reason.

The Transcript Has One Mistake

The speaker says:

"Imagine the model allows a maximum token length of four characters."

This is **not how modern tokenizers work**.

Modern tokenizers do **not** split words because of a fixed character limit.

Instead, they use algorithms like

- Byte Pair Encoding (BPE)
- WordPiece
- SentencePiece
- Unigram

They split based on **learned frequent patterns**, not character count.

Example

internationalization



might become

international
ization



or

inter
national
ization



not every four characters.

Token IDs

After tokenization, every token becomes an integer.

Example

Have



↓

Token ID = 1423

the

↓

Token ID = 279

preced

↓

Token ID = 8142

ed

↓

Token ID = 291

The model never sees the text directly.

It only sees

```
[1423, 279, 8142, 291]
```



Embeddings

Neural networks cannot understand token IDs directly.

Each ID is converted into a dense vector.

Example

```
1423
```



```
↓
```

```
[0.12,  
-0.88,  
0.45,  
...  
0.17]
```

Usually

- 768 dimensions
- 1024 dimensions
- 4096 dimensions
- 8192 dimensions

depending on the model.

Now the model has meaningful numerical representations.

Transformer

The embedding vectors enter the Transformer.

```
Embeddings
```



```
↓
```


Self Attention



Feed Forward



Attention



Feed Forward



...



Output

Attention lets each token "look at" every other token.

Example

The cat sat on the mat.



When processing

sat



the model pays attention to

cat



rather than

the



because "cat" is more relevant.

Next Token Prediction

Finally, the model predicts the next token.

Example

Input

The capital of France is



The model computes probabilities

Paris	0.94
London	0.02
Berlin	0.01
Rome	0.01



It selects the next token

Paris



Then repeats the process until the response is complete.

Complete Pipeline

User Prompt



Text



Tokenizer



Token IDs



Embedding Layer



Dense Vectors



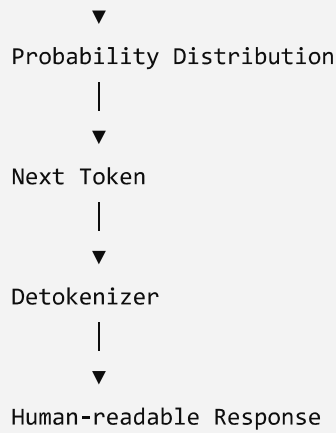
Positional Encoding



Transformer Layers

(Self Attention + Feed Forward)





Why do modern LLMs prefer subword tokenization?

- **Smaller vocabulary:** No need to store every possible word.
- **Handles unseen words:** New words can be broken into known subword pieces.
- **Supports multiple languages:** Common roots and suffixes are reused across languages.
- **More efficient:** It strikes a balance between very long character sequences and enormous word vocabularies.

This is why most modern LLMs—such as GPT, Llama, Gemma, Qwen, Mistral, and Claude—use **subword tokenization** (via algorithms like BPE, SentencePiece, or WordPiece) rather than pure word- or character-level tokenization.

